Grogoni June doc 1

Grogoni Architecture v3.0

Master Architecture Document

Document Status

Document Title: Grogoni Architecture v3.0
Document Type: Master Product and Solution Architecture
Platform: Grogoni
Runtime: Grogoniclaw
Primary Target Users: Informal businesses, small retailers, agribusinesses, suppliers, sales agents, logistics actors, and low-resource market operators
Primary Deployment Target: Low-end Android smartphones through a mobile-first web application/PWA
Secondary Deployment Target: Cloud-assisted AI and sync infrastructure
Document Purpose: Define the full business, product, data, AI, runtime, sync, integration, marketplace, and deployment architecture for Grogoni.

---

1. Executive Summary

Grogoni is a mobile-first AI-native business operating system designed for informal and small enterprises operating in low-resource environments. It enables shop owners, suppliers, sales agents, logistics operators, and customers to manage commerce through a conversational AI interface instead of complex dashboards.

The system is designed around four foundational ideas:

1. **Mobile-first commerce**
   The entire platform must work on low-end smartphones with small screens, weak processors, limited RAM, expensive data, and unstable connectivity.

2. **AI-native business operation**
   Users interact with an AI agent that understands business commands, fills structured templates, manages records, and executes workflows.

3. **Model templates plus orchestration**
   Business structures such as products, customers, invoices, suppliers, logistics, payments, and taxes are represented as templates. The orchestrator fills these templates using data from chats, files, contacts, APIs, camera, GPS, and connected accounts.

4. **Local-first, cloud-assisted intelligence**
   Small local models handle lightweight tasks on-device, while cloud models assist with heavy reasoning, document processing, verification, and marketplace operations when necessary.

Grogoni is not simply an e-commerce app. It is an AI-native business operating system for informal markets.

---

2. Product Definition

Grogoni is a mobile-first AI-native business network application that allows users to run small businesses through an AI agent.

In practical terms, the platform helps users manage:

- Products
- Inventory
- Customers
- Suppliers
- Sales agents
- Invoices
- Payments
- Deliveries
- Taxes
- Credit and debt
- Documents
- Business records
- Communication
- Reports
- Marketplace skills

The user does not need to understand databases, accounting software, dashboards, ERP systems, or APIs. The user communicates with the agent through chat.

Example user commands:

Add 5 crates of tomatoes.
Create invoice for John.
Who owes me money?
Send this invoice to WhatsApp.
Update buying price of maize to KES 3,000.
Show today's deliveries.
Remind customers who have not paid.
Import products from this spreadsheet.

The AI agent interprets the command, maps it to the correct business template, checks permissions, retrieves or updates data, verifies the action, and responds in natural language.

---

3. Mission

The mission of Grogoni is:

To give every informal business owner a lightweight AI business operating system that runs on the phone they already own.

The platform is designed especially for markets where:

- Formal ERP software is too expensive.
- POS systems are not widely used.
- Businesses run through WhatsApp, phone calls, notebooks, memory, and trust.
- Internet is unreliable.
- Smartphones are low-end.
- Business records are scattered across contacts, messages, photos, PDFs, spreadsheets, notebooks, and memory.
- Customers buy on credit.
- Payments happen through mobile money.
- Deliveries are informal and flexible.
- Suppliers and agents operate through personal relationships.

Grogoni converts these fragmented business practices into structured, AI-readable, syncable business data.

---

4. Core Design Principles

4.1 Mobile-First

The first design target is not desktop. It is a low-end Android phone.

The app must support:

- Small screens
- Limited RAM
- Weak CPUs
- Slow storage
- Intermittent internet
- Expensive data bundles
- Low battery availability
- Rural and peri-urban usage
- Informal business workflows
- Non-technical users

The system should be usable while standing in a shop, market, farm, delivery point, kiosk, or roadside stall.

4.2 Low-End Smartphone First

The platform must be able to function on phones with:

- 1 GB RAM minimum target
- Limited storage
- Low network bandwidth
- Older Android browsers
- Low processing capacity
- Unstable power supply

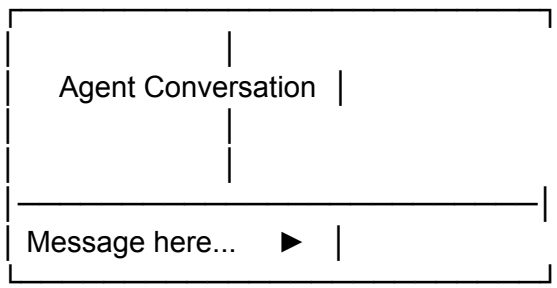
This means:

- The PWA shell must be lightweight.
- The chat UI must load fast.
- Background sync must be controlled.
- Local models must be optional.
- Heavy AI tasks must fall back to the cloud.
- Forms must be minimal.
- Offline cache must be prioritized.
- Image upload should be compressed.
- Document processing can be deferred.

4.3 Chat-First Interface

The main interface is not a dashboard.

The main interface is conversation.



The agent opens forms only when necessary.

Example:

User: Add maize.
Agent: How many bags?
User: 20.
Agent: Buying price?
User: 3000.
Agent: Selling price?
User: 3500.
Agent: Saved 20 bags of maize.

This keeps the platform simple for users who are unfamiliar with formal software.

4.4 Offline-First

The system must not stop working when internet fails.

Core offline functions:

- Add product
- Add customer
- Create invoice
- Record sale
- Record payment manually
- View existing records
- Prepare delivery
- Save draft message
- Queue sync action

When connectivity returns, the sync engine uploads pending changes.

4.5 AI-Assisted, Not AI-Uncontrolled

The AI agent may suggest, prepare, calculate, summarize, classify, and recommend actions.

However, sensitive actions require confirmation.

Examples requiring confirmation:

- Sending money
- Deleting invoices
- Submitting tax records
- Sending messages to many customers
- Changing product prices
- Modifying stock quantities
- Linking external accounts
- Importing large datasets
- Installing marketplace skills

The AI does not directly control business-critical operations without the user's approval.

4.6 Local-First, Cloud-Assisted

The app should work locally first.

Cloud is used for:

- Sync
- Backup
- Large document processing
- Heavy AI reasoning
- Marketplace
- Model updates
- Verification
- Cross-device access
- Payment webhooks
- Tax integrations

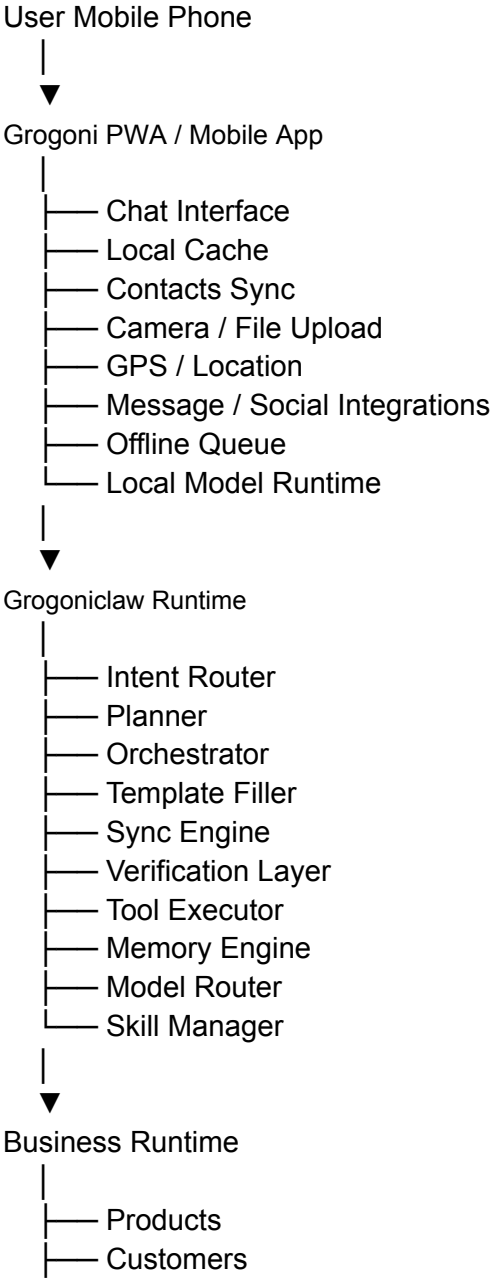
Local device is used for:

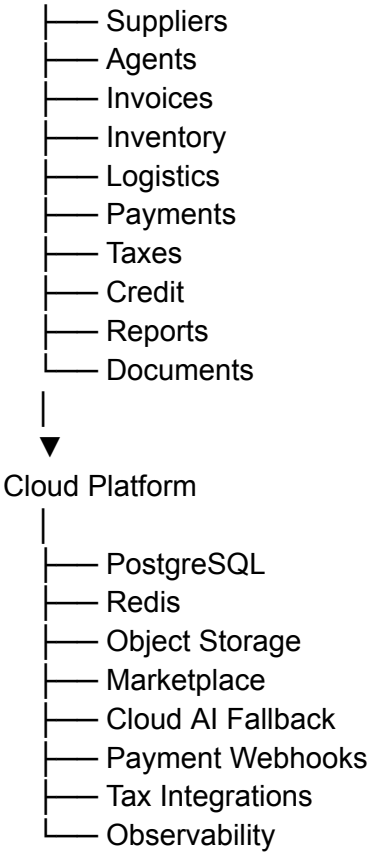
- Chat interface
- Offline cache
- Basic command parsing

- Small local model inference where available
- Local drafts
- Quick search
- Basic invoice generation
- Temporary queueing

---

5. High-Level Platform Architecture





---

6. Platform Runtime Separation

Grogoni should be separated into four major runtimes.

Application Runtime
Business Runtime
AI Runtime
Device Runtime

This separation prevents the platform from becoming a tightly coupled monolith.

6.1 Application Runtime

The Application Runtime handles the user-facing app.

Responsibilities:

- Login
- Onboarding
- Chat UI
- Settings
- Notifications
- Local navigation
- Offline state
- Sync status
- User confirmations
- Permission prompts

It should be lightweight and responsive even on weak phones.

6.2 Business Runtime

The Business Runtime handles real business logic.

It owns:

- Product rules
- Inventory rules
- Customer records
- Supplier records
- Invoice generation
- Credit tracking
- Payment records
- Delivery records
- Tax calculation
- Reports
- Audit logs

The Business Runtime should not depend on the AI. The AI calls business APIs, but the business rules remain deterministic.

6.3 AI Runtime

The AI Runtime is Grogoniclaw.

It handles:

- Intent detection
- Planning
- Orchestration
- Tool selection

- Template filling
- Model routing
- Memory
- Verification
- Skill execution
- Response generation

The AI Runtime does not directly mutate critical data. It proposes structured actions which pass through the Business Runtime and Verification Layer.

6.4 Device Runtime

The Device Runtime handles phone capabilities.

It manages access to:

- Contacts
- Camera
- Microphone
- GPS
- File manager
- SMS
- Calendar
- Notifications
- Local storage
- Network state
- Battery state

The Device Runtime must always respect user permissions.

---

7. Model Templates and Orchestration

7.1 Model Templates

Model templates are structured business schemas.

They define what a record should look like, but they do not store actual user data.

Example:

Product Template

- Name
- Code
- Description
- Quantity
- Buying Price
- Selling Price
- Unit of Measurement
- Image
- Packaging

A template is reusable across businesses.

Templates exist for:

- Customer
- Supplier
- Agent
- Product
- Invoice
- Logistics
- Payment
- Tax
- Credit
- Return
- Pricing Rule
- Delivery Route
- Document Upload
- Skill
- Report

7.2 Orchestration

The orchestrator fills templates using real data.

Data sources include:

- User chat
- Contact list
- Uploaded documents
- Existing databases
- Camera
- GPS
- Gmail

- Calendar
- WhatsApp
- Facebook
- Instagram
- X
- LinkedIn
- Payment APIs
- Logistics APIs
- Government APIs
- Manual entry
- Previous records
- Business memory

Example:

```
Template:
Customer {
  name,
  contact,
  location
}

Filled Record:
Customer {
  name: "John Mwangi",
  contact: "+2547...",
  location: "Machakos Town"
}
```

7.3 Why Templates Matter

Templates allow Grogoni to support many business types without rebuilding the system.

For example:

A poultry business, a cereal shop, a hardware store, and a pharmacy may all use the same core Product Template, but each may install different skill packs and field extensions.

---

8. Core Data Models

8.1 Account

Account
- id
- business_name
- owner_user_id
- phone_number
- email
- country
- base_currency
- language
- verification_tier
- subscription_plan
- created_at
- updated_at

An Account represents the business entity.

8.2 User

User
- id
- account_id
- name
- phone
- email
- role
- login_method
- status
- created_at
- updated_at

A business account may have many users.

8.3 Agent

Agent
- id
- account_id
- name
- role
- model_config
- permissions

- connected_tools
- installed_skills
- memory_scope
- status
- created_at
- updated_at

Agent examples:

- Shop attendant
- Sales agent
- Supplier assistant
- Logistics assistant
- Inventory assistant
- Customer support agent

8.4 Customer

Customer
- id
- account_id
- name
- phone
- email
- address
- gps_location
- source
- credit_status
- notes
- created_at
- updated_at

Customer sources:

- Manual entry
- Contact list sync
- WhatsApp chat
- Invoice
- Delivery record
- Uploaded file
- Social media message

8.5 Supplier

Supplier
- id
- account_id
- name
- phone
- email
- address
- products_supplied
- payment_terms
- notes
- created_at
- updated_at

8.6 Product

Product
- id
- account_id
- supplier_id
- code
- name
- description
- quantity
- reorder_threshold
- reorder_quantity
- buying_price
- selling_price
- unit_of_measurement
- image_url
- packaging
- source
- status
- created_at
- updated_at

Product sources:

- Manual entry
- Existing database
- Uploaded document
- Spreadsheet

- PDF
- Word file
- Global product list template
- Camera/photo
- Supplier catalog

8.7 Inventory Movement

InventoryMovement
- id
- account_id
- product_id
- movement_type
- quantity
- reason
- related_invoice_id
- related_purchase_id
- performed_by
- created_at

Movement types:

- Stock in
- Sale
- Return
- Damage
- Adjustment
- Transfer
- Expiry
- Loss

8.8 Invoice

Invoice
- id
- account_id
- invoice_number
- customer_id
- supplier_id
- agent_id
- route
- date
- due_date

- terms
- location
- contact
- items
- subtotal
- tax_amount
- discount_amount
- total_amount
- payment_status
- delivery_status
- created_at
- updated_at

8.9 Invoice Item

InvoiceItem
- id
- invoice_id
- product_id
- description
- quantity
- unit_price
- discount
- tax_amount
- line_total

8.10 Payment

Payment
- id
- account_id
- invoice_id
- customer_id
- amount
- currency
- method
- provider
- transaction_reference
- status
- received_at
- created_at
- updated_at

Payment providers:

- M-Pesa
- Stripe
- PayPal
- Pogo
- MoneySwitch
- Other local payment APIs

8.11 Logistics

Logistics
- id
- account_id
- vehicle
- driver_name
- driver_contact
- sales_agent_contact
- route
- departure_location
- delivery_location
- date_time
- customer_contact
- quantity
- weight
- status
- created_at
- updated_at

8.12 Tax Record

TaxRecord
- id
- account_id
- invoice_id
- tax_type
- tax_rate
- tax_amount
- authority
- status
- submitted_at

- created_at
- updated_at

8.13 Credit Account

CreditAccount
- id
- account_id
- customer_id
- credit_limit
- current_balance
- due_date
- interest_rate
- status
- created_at
- updated_at

8.14 Credit Note

CreditNote
- id
- account_id
- original_invoice_id
- reason
- items_returned
- refund_amount
- refund_method
- status
- created_at
- updated_at

8.15 Pricing Rule

PricingRule
- id
- account_id
- product_id
- customer_tier
- rule_type
- condition
- price

- commission_rate
- start_date
- end_date
- status

Supported pricing rules:

- Bulk pricing
- Agent commission
- Customer-tier pricing
- Promotional pricing
- Seasonal pricing
- Wholesale pricing
- Retail pricing

8.16 Vocabulary Alias

VocabAlias
- id
- account_id
- alias
- canonical_type
- canonical_id
- confidence
- created_at
- updated_at

Example:

Alias: "unga ya ugali"
Canonical Product: Maize flour

Aliases are account-scoped.

---

9. Business Logic Architecture

9.1 Inventory Logic

Inventory must be updated through controlled movements.

Rules:

- A sale decreases inventory.
- A return increases inventory.
- A damaged item decreases usable inventory.
- Manual adjustment requires reason.
- Stock below threshold triggers an alert.
- Stock cannot silently become negative unless the owner allows backorders.

9.2 Invoice Logic

Invoices should support:

- Draft
- Issued
- Partially paid
- Paid
- Overdue
- Cancelled
- Refunded

Invoice creation should check:

- Customer existence
- Product existence
- Available quantity
- Pricing rules
- Tax rules
- Credit limits
- Agent permissions

9.3 Credit and Debt Logic

Informal businesses depend heavily on credit.

The system must answer:

Who owes me money?
What does John still owe?
Which invoices are overdue?
Who should I remind today?

Credit logic includes:

- Customer balances
- Due dates
- Partial payments
- Credit limits
- Overdue reminders
- Credit history
- Risk flags

9.4 Returns and Refunds

Returns are handled through Credit Notes.

Return flow:

Original Invoice
↓
Return Request
↓
Credit Note
↓
Inventory Restock
↓
Balance Adjustment
↓
Refund or Credit

9.5 Pricing Rules

A single selling price is not enough.

The system supports:

- Retail price
- Wholesale price
- Bulk discount
- Customer-specific price
- Agent commission
- Promotional price
- Time-limited price
- Region-specific price

9.6 Tax Logic

Tax rules are configurable by country.

TaxRule
- country
- tax_type
- rate
- applies_to
- effective_date
- expiry_date

The system calculates tax at invoice time and stores a snapshot so old invoices remain historically correct.

---

10. Grogoniclaw Runtime Architecture

Grogoniclaw is the AI execution brain of Grogoni.

High-level flow:

User Message
↓
Intent Detection
↓
Planning
↓
Template Selection
↓
Data Retrieval
↓
Tool Execution
↓
Business Rule Validation
↓
Verification
↓
Sync
↓
Memory Update

↓
Response

10.1 Runtime Components

Grogoniclaw Runtime
- Intent Router
- Planner
- Task Scheduler
- Orchestrator
- Template Registry
- Tool Executor
- Model Router
- Memory Engine
- Knowledge Engine
- Verification Layer
- Permission Manager
- Skill Manager
- Response Generator

10.2 Intent Router

Classifies user messages into intents.

Examples:

- Add product
- Update product
- Create invoice
- Record payment
- Check debt
- Send message
- Upload document
- Create delivery
- Link account
- Generate report

If confidence is low:

- Confidence below 0.5: ask clarifying question.
- Confidence below 0.3: show common actions.
- Three failures: offer structured form.

10.3 Planner

The planner breaks intents into steps.

Example:

User: Create invoice for John for 2 bags of maize.

Plan:
1. Find customer John.
2. Find product maize.
3. Check stock.
4. Apply pricing.
5. Calculate tax.
6. Create invoice.
7. Ask whether to send it.

10.4 Tool Executor

The Tool Executor performs actions through controlled APIs.

Examples:

- create_product
- update_product
- create_invoice
- record_payment
- send_message
- import_document
- sync_contacts
- calculate_tax
- generate_report

No tool executes without permission checks.

10.5 Verification Layer

Verification checks:

- User permission
- Agent permission
- Business rules
- Stock availability

- Credit limits
- Tax compliance
- Fraud signals
- Payment status
- Required confirmation

10.6 Response Generator

The response generator explains what happened in simple language.

Example:

Invoice INV-0042 created for John.
Total: KES 7,000.
Payment status: Unpaid.
Would you like me to send it by SMS or WhatsApp?

---

11. Runtime Execution Architecture

11.1 Standard Execution Flow

User Input
↓
Normalize Text
↓
Detect Language
↓
Detect Intent
↓
Build Context
↓
Generate Plan
↓
Select Tools
↓
Check Permissions
↓
Execute Tools
↓
Verify Results

↓
Update Memory
↓
Queue Sync
↓
Respond

11.2 Example: Create Invoice

User: Create invoice for John, 2 bags maize.

1. Intent Router detects create_invoice.
2. Context Builder retrieves customer John.
3. Product Resolver finds maize.
4. Inventory Engine checks stock.
5. Pricing Engine applies price.
6. Tax Engine calculates tax.
7. Invoice Service creates invoice.
8. Sync Engine queues cloud update.
9. Agent responds with invoice summary.

11.3 Example: Import Product List

User uploads spreadsheet.

1. Document Upload Engine receives file.
2. Parser detects columns.
3. Field Extractor maps columns to Product Template.
4. Agent shows preview.
5. User confirms.
6. Product records are saved.
7. Inventory records are created.
8. Sync queue uploads records.

11.4 Example: Low Confidence

User: Add John maize.

Possible meanings:
- Add John as customer.
- Add maize sold to John.

- Add maize supplied by John.

Agent: Did you mean create a sale/invoice for John or add John as a customer?

---

12. Event-Driven Architecture

Everything important should emit events.

Examples:

ProductCreated
ProductUpdated
StockAdjusted
CustomerCreated
InvoiceCreated
PaymentReceived
DeliveryCreated
DeliveryCompleted
CreditLimitExceeded
TaxCalculated
DocumentImported
SkillInstalled

12.1 Event Flow Example

InvoiceCreated
  ↓
InventoryUpdated
  ↓
TaxRecordCreated
  ↓
CustomerBalanceUpdated
  ↓
ReportUpdated
  ↓
SyncQueued
  ↓
NotificationPrepared

12.2 Why Events Matter

Events allow:

- Offline sync
- Audit logs
- Marketplace skills
- Reports
- Notifications
- Fraud detection
- Rollbacks
- Integrations

Each event should be immutable.

---

13. Knowledge Architecture

Grogoni has three layers:

Templates
↓
Business Data
↓
Knowledge

13.1 Templates

Templates define structure.

Example:

Product Template
Customer Template
Invoice Template

13.2 Business Data

Business data stores facts.

Example:

Product: Maize
Quantity: 20 bags
Selling price: KES 3,500

13.3 Knowledge

Knowledge stores insights derived from data.

Examples:

- Top-selling products
- Slow-moving products
- Best customers
- Credit risk customers
- Seasonal trends
- Reorder recommendations
- Sales forecasts
- Profit trends
- Delivery performance
- Agent performance

This allows the AI to answer strategic questions.

Example:

User: What should I restock this week?

Agent: Maize flour and cooking oil are moving fastest. Maize flour has only 4 bags left and usually sells 8 bags per week. I recommend restocking at least 10 bags.

---

14. Memory Architecture

Grogoni memory is divided into five types.

14.1 Conversation Memory

Stores recent dialogue.

Used for:

- Context continuity
- Clarification
- Current task tracking
- Recent entities mentioned

14.2 Business Memory

Stores business preferences.

Examples:

- Preferred invoice format
- Default payment method
- Common suppliers
- Operating hours
- Default tax settings

14.3 Customer Memory

Stores customer-specific patterns.

Examples:

- Usually buys on credit
- Prefers SMS
- Pays weekly
- Common delivery location

14.4 Knowledge Memory

Stores summaries and insights.

Examples:

- Monthly sales summary
- Stock movement summary
- Debt summary
- Agent performance summary

14.5 Skill Memory

Stores skill-specific settings.

Examples:

- Poultry skill feed conversion assumptions
- Pharmacy skill expiry rules
- Hardware skill unit conversion rules

---

15. Model Registry

Grogoni supports one agent using multiple models.

Model Registry
├── Tiny Local Model
├── Small Local Model
├── Vision Model
├── Speech Model
├── Reasoning Model
├── Extraction Model
└── Cloud Model

15.1 Model Selection Criteria

The Model Router selects a model based on:

- Device RAM
- CPU capacity
- Battery level
- Network availability
- Task complexity
- Required accuracy
- Cost
- Latency
- Privacy requirement

15.2 Model Usage Examples

Tiny local model:

- Command classification
- Simple extraction
- Offline chat shortcuts

Small local model:

- Basic reasoning
- Product entry
- Invoice assistance
- Customer lookup

Vision model:

- Product photo understanding
- Receipt extraction
- Document image parsing

Cloud model:

- Complex reasoning
- Large document extraction
- Cross-document analysis
- Tax report generation
- Skill verification

---

16. AI Skills Architecture

Skills extend the Grogoni runtime.

Core Runtime
↓
Skill Manager
↓
Installed Skills
↓
Marketplace

Examples:

- POS Skill
- Poultry Skill

- Pharmacy Skill
- Hardware Store Skill
- Agribusiness Skill
- Delivery Skill
- Tax Skill
- Hotel Skill
- Wholesale Skill
- Credit Management Skill

16.1 Skill Permissions

Skills must request permissions.

Examples:

Read products
Write invoices
Read customers
Send messages
Access payments
Generate reports

Skills cannot access raw databases directly.

They must use Tool Executor APIs.

16.2 Skill Safety

Marketplace skills must be:

- Sandboxed
- Versioned
- Signed
- Permission-scoped
- Auditable
- Revocable
- Testable
- Rollback-ready

---

17. Business Digital Twin

Each business has a digital twin.

Real Business
↓
Business Records
↓
Business Twin
↓
AI Analysis
↓
Recommendations

The digital twin represents:

- Products
- Stock
- Customers
- Suppliers
- Debt
- Cash flow
- Sales
- Logistics
- Payments
- Taxes
- Staff activity

17.1 Digital Twin Uses

The agent can answer:

What happens if I increase maize price by 10%?
Which customers are becoming risky?
Which product should I stop buying?
How much cash should I expect this week?
What stock will run out first?

17.2 Simulation

The digital twin allows what-if analysis.

Examples:

- Price increase
- Bulk purchase
- Supplier change
- Delivery delay
- Credit tightening
- Promotion campaign
- Seasonal demand shift

---

18. API Architecture

18.1 Internal APIs

/auth
/accounts
/users
/agents
/customers
/suppliers
/products
/inventory
/invoices
/payments
/logistics
/taxes
/credit
/documents
/sync
/templates
/tools
/events
/skills
/marketplace
/reports

18.2 External API Connectors

Contacts API
WhatsApp API

Instagram API
Facebook API
X API
LinkedIn API
Gmail API
Calendar API
Payment APIs
Logistics APIs
Government Tax APIs
SMS Gateway
USSD Gateway

18.3 Tool API Pattern

Every external action is wrapped as a tool.

Example:

Tool: create_invoice

Input:
- customer
- products
- quantity
- price
- terms

Output:
- invoice_number
- invoice_url
- payment_link

---

19. Sync Engine

The sync engine keeps local and cloud data consistent.

19.1 Offline-First Sync Flow

Local Action

↓
Save to Local Database
↓
Create Event
↓
Add to Sync Queue
↓
Check Connectivity
↓
Upload Delta
↓
Resolve Conflict
↓
Confirm Sync

19.2 Sync Sources

The app syncs with:

- Contact list
- Social media apps
- Messages
- Gmail
- Calendar
- Camera
- Microphone
- File manager
- GPS
- Payment providers
- Existing databases
- Cloud account
- Marketplace skills

19.3 Conflict Resolution

Rules:

1. Prefer latest confirmed user action.
2. Preserve both versions if uncertain.
3. Ask user when money, quantity, tax, or invoice data conflicts.
4. Never overwrite payment records silently.
5. Keep audit logs of all conflict resolutions.

---

20. Document Upload Engine

Users can upload:

- PDF
- Word documents
- Spreadsheets
- Images
- Product lists
- Supplier catalogs
- Receipts
- Invoices

20.1 Processing Flow

Uploaded File
↓
File Type Detection
↓
Document Parser
↓
Field Extractor
↓
Template Mapper
↓
Preview
↓
User Confirmation
↓
Database Save
↓
Sync Queue

20.2 Example

Spreadsheet row:

Maize | 20 bags | KES 3,000 | KES 3,500

Mapped record:

Product:
- name: Maize
- quantity: 20 bags
- buying_price: 3000
- selling_price: 3500

---

21. Permission Architecture

The agent must request permission before accessing:

- Contacts
- Camera
- Microphone
- GPS
- File manager
- Gmail
- Calendar
- Social accounts
- Payment accounts
- SMS
- Marketplace skills

21.1 Role-Based Access

Owner:

- Full access

Manager:

- Operations access
- No billing or deletion

Sales Agent:

- Customers
- Invoices
- Logistics
- No price editing unless allowed

Cashier:

- Payments
- Receipts

Supplier:

- Product and delivery access

Customer:

- Own invoices and orders only

View-only:

- Reports and read access

---

22. Security Architecture

22.1 Core Rules

- Encrypt sensitive records.
- Store payment credentials securely.
- Use OAuth/API linking for external services.
- Never expose user tokens to the AI model.
- Separate AI reasoning from actual API execution.
- Require confirmation for sensitive operations.
- Log all agent actions.
- Maintain audit trails.
- Use scoped permissions for tools and skills.

22.2 Sensitive Actions

Require confirmation for:

- Send payment
- Delete invoice
- Change product quantity
- Change pricing rules
- Send message to customer
- Sync contacts

- Connect social media
- Submit tax record
- Install marketplace skill
- Export business data

---

23. Compliance Architecture

23.1 Merchant Verification

Verification tiers:

Tier 0: Phone only
Tier 1: Phone + National ID
Tier 2: Business registration
Tier 3: Full KYB

Unverified users can still use basic records, but payments, tax, and cross-border features may be gated.

23.2 Tax Compliance

Tax rules are configurable by country.

Supported authorities may include:

- KRA
- URA
- TRA
- Other regional tax authorities

The system should support:

- VAT
- Withholding tax
- Excise
- Exempt goods
- Monthly tax summaries
- Digital fiscal compliance where required

23.3 Data Residency

Each account has:

DataResidencyConfig
- home_country
- allowed_regions
- consent_status

Personal and business data should not be moved across jurisdictions without consent where local law restricts it.

23.4 Data Retention

Suggested defaults:

- Invoice/payment records: 7 years
- Chat history: 30 days default
- Reports: retained while account is active
- Deleted accounts: anonymize business records where audit preservation is needed
- User export: available as ZIP with JSON and PDFs

---

24. Connectivity Architecture

24.1 Data Connectivity

The app should work with:

- Full internet
- Weak internet
- Intermittent internet
- Data-off periods
- SMS fallback
- USSD fallback for critical confirmations

24.2 SMS Fallback

SMS can support:

- Invoice summaries
- Payment links
- Stock alerts
- Payment confirmation parsing
- Customer reminders

24.3 USSD Fallback

USSD can support simple actions:

- Confirm delivery
- Check balance
- Request invoice
- Confirm payment
- View outstanding amount

USSD should remain under five steps.

---

25. Resource Budget

Target constraints:

PWA shell: under 15 MB
Chat UI: usable on 1 GB RAM devices
Local model: optional
Background sync: maximum 1 MB per cycle
Battery: no aggressive background processes
Sync frequency: no more than every 15 minutes unless user-triggered
Image upload: compressed by default
Document upload: defer heavy processing to cloud

---

26. Language and Dialect Support

The platform must support multilingual markets.

Initial target languages:

- English
- Swahili
- Sheng support
- Luganda
- Amharic
- French for future regional expansion

Input support:

- Mixed-language chat
- Local product names
- Local date terms
- Local price formats
- Abbreviations
- Informal spellings

Examples:

leo = today
kesho = tomorrow
3.5K = 3500
unga ya ugali = maize flour

---

27. Customer Self-Service Portal

Customers should receive lightweight links.

They can:

- View invoice
- Pay invoice
- Confirm delivery
- Request quote
- Reorder
- View payment status

No separate app is required.

A token-authenticated mini web view is enough for MVP.

---

28. Fraud and Trust Signals

The system should detect:

- Repeated payment reversals
- Unusual invoice amounts
- Agent actions above threshold
- Suspicious bulk deletion
- High credit exposure
- Delivery disputes
- Frequent refund requests

The system should alert the owner before executing risky actions.

---

29. Onboarding Architecture

Target:

First completed business action within 3 minutes.

Flow:

1. Phone number + OTP
2. Business name
3. Business type
4. Language selection
5. Agent introduction
6. Add first product
7. Create first stock record or invoice

No long tutorials.

Learning happens through doing.

---

30. Platform Pricing Architecture

Suggested pricing:

Free Tier
- 1 user
- 50 products
- 20 invoices/month
- Basic AI commands

Business Tier
- KES 500/month
- Unlimited products
- 5 staff accounts
- Document upload
- Sync

Pro Tier
- KES 1,500/month
- Logistics
- Full integrations
- Tax reports
- Priority cloud AI

Transaction Fee Option
- 0.5% on processed payments

Marketplace Revenue
- 20% Grogoni
- 80% developer

---

31. Marketplace Architecture

The marketplace can distribute:

- Skills
- Templates
- Agents
- Workflows
- Industry packs
- Prompt libraries
- Business reports
- Integrations

Examples:

Poultry Pack
Hardware Shop Pack
Pharmacy Pack
Agrovet Pack
Cereal Shop Pack
Fast Food Pack
Delivery Pack
Tax Filing Pack

Each marketplace item must be:

- Versioned
- Signed
- Permission-scoped
- Reviewable
- Rollback-ready
- Auditable

---

32. Development Roadmap

Phase 1: Mobile-First MVP

Build:

- PWA
- Phone/email login
- Chat interface
- Product management
- Customer management
- Basic invoice generation
- Local storage
- Manual entry
- Basic command parser

Goal:

A shop owner can add products, add customers, and generate invoices from chat.

Phase 2: Template System

Build:

- Product template
- Customer template
- Supplier template
- Invoice template
- Logistics template
- Payment template
- Agent template
- Template registry
- Template mapper

Goal:

Grogoni converts user input into structured business records.

Phase 3: Document Upload Engine

Build:

- Spreadsheet upload
- PDF upload
- Word upload
- Product extraction
- Supplier catalog import
- User confirmation flow

Goal:

Businesses can import existing records.

Phase 4: Offline Sync

Build:

- Local cache
- Cloud sync
- Conflict resolution
- Background sync
- Sync queue

- Retry logic

Goal:

The app works with poor internet.

Phase 5: External Integrations

Build:

- Contacts sync
- Gmail linking
- Calendar linking
- WhatsApp integration
- Facebook/Instagram integration
- X/LinkedIn integration
- File manager
- GPS
- Camera

Goal:

The agent can use phone and social data.

Phase 6: Payments

Build:

- M-Pesa integration
- Stripe integration
- PayPal integration
- Payment confirmation
- Invoice status
- Transaction records

Goal:

Users can track and process payments.

Phase 7: Logistics

Build:

- Vehicle records
- Driver records
- Route planning
- Delivery status
- Customer delivery contact
- Quantity and weight tracking

Goal:

Users can manage product movement.

Phase 8: Local AI Runtime

Build:

- llama.cpp wrapper
- Device capability detection
- Small model installation
- Local inference
- Cloud fallback
- Model router

Goal:

Grogoni runs useful AI features on low-end smartphones.

Phase 9: Marketplace

Build:

- Skill packaging
- Skill installation
- Skill leasing
- Skill selling
- Certification badge
- Performance dashboard
- Versioning and rollback

Goal:

Developers and businesses can create, sell, lease, and install AI business skills.

33. Final System Vision

Grogoni should become:

A mobile-first AI business operating system for informal and small enterprises, powered by lightweight local models, structured business templates, syncable business data, and AI agents that help users run commerce through conversation.

The final platform combines:

AI Agent
+ Grogoniclaw Runtime
+ Business Templates
+ Business Digital Twin
+ Mobile Phone Data
+ Local SLMs
+ Cloud AI Fallback
+ Payment APIs
+ Logistics APIs
+ Social Media APIs
+ Tax APIs
+ Offline Sync
+ SMS/USSD Fallback
+ Marketplace Skills

The result is not just an app.

It is an AI-native business network for low-resource markets.